

Day 7 — Databricks, Delta Lake, ETL/ELT, MLflow, and data lifecycle

Outcome

Be able to explain how raw enterprise data becomes governed tables, ML features, RAG knowledge, reproducible experiments, approved models, deployed endpoints, and auditable lineage.

1. Databricks and the lakehouse

Databricks is a unified data and AI platform for ingestion, processing, analytics, ML, GenAI, automation, and governance.

data lake flexibility
+ warehouse reliability
= lakehouse

Core components

Component	Role
Workspace	Organizes notebooks, jobs, queries, experiments, dashboards, and assets.
Notebook	Interactive Python/SQL/Scala/R and narrative work.
All-purpose compute	Interactive development.
Job compute	Automated production work with controlled lifecycle/cost.
Serverless compute	Managed execution where the platform handles more of the compute lifecycle.
SQL warehouse	SQL analytics and dashboards.
Jobs/pipelines	Scheduled or event-driven task orchestration.
Delta Lake	Reliable table/storage foundation.
MLflow	Experiment, model, and GenAI lifecycle.
Unity Catalog	Access control, lineage, discovery, audit, and governance.
Apps/Agents	Data/AI applications and tool-using experiences.

The source notes use both “cluster” and “compute.” Treat compute as the broader execution category; all-purpose and job clusters are cluster-backed examples, while serverless offerings abstract more cluster management.

Notebooks are useful for exploration and development; production pipelines require source control, tests, jobs, dependency control, and monitoring.

Databricks Apps provide application experiences backed by Databricks data and APIs. Databricks Agents add systems that retrieve, reason, call tools, and complete multi-step tasks, with the same need for permissions, evaluation, tracing, and lifecycle control.

Ingestion and production execution

Databricks can ingest files/object storage, databases, streams, and APIs. A production flow separates exploratory notebooks from repeatable jobs:

```
source
→ ingest raw data with source/time/batch metadata
→ write Bronze Delta table
→ job transforms and validates Silver
→ job publishes Gold analytics/features or RAG-ready data
→ monitoring and quality results
```

Source-note ingestion options include Auto Loader for incremental cloud-file ingestion, COPY INTO for SQL-driven incremental loading, and Lakeflow Spark Declarative Pipelines for managed ETL pipelines. Choose from source type, latency, state/checkpoint needs, team skills, and operational ownership rather than memorizing one universal mechanism.

Use notebooks to explore and develop the transformation. Move production logic into version-controlled, tested code executed by jobs/pipelines on appropriately scoped job compute. Use SQL warehouses for governed SQL analytics rather than treating an interactive cluster as every workload.

2. ETL, ELT, and medallion architecture

- ETL: extract → transform outside target → load clean result.
- ELT: extract → load raw data → transform inside the platform.

Databricks commonly supports ELT with:

```
Bronze → Silver → Gold
```

Bronze

- raw source representation;
- ingestion metadata;
- append/incremental capture;
- replay/audit foundation.

Do not treat Bronze as clean business data.

Silver

- cleaned and standardized;
- deduplicated;
- validated schema/ranges;
- conformed identifiers;
- usable joins;
- quality flags.

Gold

- business aggregates;
- reporting tables;
- ML-ready feature tables;
- application-serving structures.

Do not put every raw and cleaned detail into Gold.

3. Delta Lake

Delta Lake adds a transaction log and reliable table behavior to lakehouse storage.

Capabilities

- ACID transactions.
- Schema enforcement.
- Controlled schema evolution.
- Time travel/version history.
- Batch and streaming.
- UPDATE, DELETE, and MERGE/upsert.
- Scalable table metadata.

Enforcement versus evolution

- Enforcement rejects data that violates the expected schema.
- Evolution allows approved changes.

Uncontrolled evolution can silently break downstream models and queries.

Time travel

Useful for audit, debugging, reproducibility, and rollback analysis. Retention still matters; do not assume every old version exists forever.

MERGE

Use incremental upsert logic rather than rewriting complete tables when records change:

```
match business key
→ update changed row
→ insert new row
```

Make reruns idempotent.

Batch and streaming

Batch processes bounded sets periodically. Streaming processes arriving events continuously or in micro-batches.

Both require:

- schema control;
- checkpoints/state;
- deduplication;
- late/out-of-order handling;

- quality and freshness monitoring;
- idempotent sinks.

Physical design pitfalls

- Too many small files slow queries.
- High-cardinality partitioning such as by customer ID can create operational problems.
- Partition by useful filters only when justified.
- Compact/optimize where appropriate.
- Build features from governed Silver/Gold data, not raw logs.

4. Data quality and feature/RAG pipelines

Quality checks:

- required fields;
- duplicates;
- ranges;
- referential consistency;
- schema;
- freshness;
- distribution shifts.

Traditional ML:

```
sources
→ Bronze raw
→ Silver clean entities/events
→ Gold feature table
→ train/evaluate model
```

RAG:

```
documents/source metadata
→ Bronze raw documents
→ Silver parsed/cleaned/ACL-aware content
→ chunks/embeddings/index
→ evaluated published knowledge version
```

Delta tables can store document catalogs, parse outputs, quality results, and evaluation data; vector search remains the retrieval-optimized layer.

5. MLflow lifecycle

Experiment tracking

Each run records:

- parameters/configuration;
- metrics;
- artifacts;
- model;
- code and data references;

- dependencies/environment;
- timestamps and ownership.

Parameters answer “what settings?” Metrics answer “how good?” Artifacts answer “what files/results?”

Examples:

```
parameters: model type, depth, learning rate, chunk size, embedding version
metrics: F1, ROC-AUC, groundedness, faithfulness, latency, cost
artifacts: model, preprocessing pipeline, confusion matrix, prompt, RAG
report
```

Registry and versions

The registry manages approved model versions, metadata, lineage, aliases, and deployment readiness.

Aliases:

```
@candidate
@challenger
@champion
@baseline
```

Applications load an approved alias rather than an uncontrolled file such as `model_final_latest.pkl`.

Promotion

```
track experiments
→ select candidate
→ validate
→ register
→ assign challenger
→ shadow/canary/business review
→ move champion alias
→ monitor
```

Rollback

Keep the previous stable version. If quality, latency, bias, errors, or cost regress, move the production alias back, record the reason, and investigate offline.

Reproducibility and lineage

Trace:

```
production version
→ registered model
→ MLflow run
→ code/job
→ data/Delta version
→ features/preprocessing
→ parameters/metrics/artifacts
→ approval/deployment
```

Track dependencies and environment; notebook success alone does not prove reproducible production behavior.

In the Databricks lifecycle described by the source notes, MLflow integrates with notebooks, jobs, Unity Catalog, Feature Store, Model Registry, Model Serving, lakehouse data, Delta Lake, and GenAI applications or agents. Treat these as connected lifecycle surfaces rather than isolated tools.

GenAI with MLflow

Track:

- prompt/app versions;
- input and retrieved context;
- model response;
- tool calls and intermediate steps;
- citations;
- latency, tokens, and cost;
- groundedness, faithfulness, relevance;
- human feedback and trace.

Do not log unrestricted confidential content simply because tracing is useful.

6. Storage architecture

Need	Store
Raw/clean/aggregate tables	Delta Lake
Large files/models/artifacts	Object storage
Transactional app/catalog state	Relational database
Flexible documents with fixed access patterns	Document NoSQL
Similarity retrieval	Vector database/search
Cache/rate limit/transient state	Redis/key-value
Runs/models/traces/registry	MLflow plus governed backing stores

Databricks connects the data, compute, jobs, SQL, MLflow, GenAI, and governance layers; it does not remove the need to design each access pattern.

7. Example: churn platform

```
CRM + billing + support + login events
→ Bronze Delta tables
→ Silver cleaned customers/orders/tickets/logins
→ Gold churn features
→ train several models
→ MLflow parameters/metrics/artifacts
→ register best candidate
→ validation and challenger
→ champion serving endpoint or batch scoring
→ monitor drift, model/system/business metrics
→ retrain or roll back
```

The model version links back to exact data and feature versions.

8. Example: enterprise RAG platform

```
wiki/files/APIs
→ incremental ingestion job
→ raw source/version/ACL capture
→ parse/clean/deduplicate
→ governed chunks and metadata
→ embedding/index build
→ golden retrieval evaluation
→ publish index version
→ trace queries, context, answers, citations, feedback
```

Changes to parser, chunker, embedding, metadata, index, prompt, or model are versioned and evaluated.

9. Common mistakes

- Treating Databricks as only Spark.
- Confusing a data lake with a governed lakehouse.
- Running production solely as manual notebooks.
- Leaving interactive compute running without cost control.
- Ignoring Unity Catalog and lineage.
- Using Bronze directly for reports/models.
- Skipping quality checks.
- Allowing uncontrolled schema changes.
- Full overwrite instead of incremental MERGE.
- Bad partitions and small-file accumulation.
- Logging only accuracy or only final answers.
- Registering every poor experiment.
- Confusing tracking, registry, serving, and monitoring.
- No data version, dependency record, or rollback target.

Project-grounded examples

Scenario 1: raw benchmark output to comparison-ready data

Project scenario. DPDK Automation for Network Packet Processing collected raw DPDK output and CPU/system statistics, used Bash scripts for parallel collection, processed measurements with a Python module, applied workload-specific parsers for testpmd, crypto, and vhost, stored structured results in a database, and exposed graphs and side-by-side run comparisons.

How the lifecycle concepts apply. Although the project did not use Databricks or Delta Lake, its data flow maps naturally to lifecycle layers:

```
raw benchmark logs and statistics
→ normalized benchmark metrics plus run/configuration context
→ comparison-ready data and dashboards
```

The raw artifacts are replay/debug evidence; the normalized layer resolves workload-specific formats; the serving layer supports comparisons such as SMT on/off, BIOS or OS changes, compiler differences, and CPU SKU variations. This is the same separation of raw, validated, and consumption-ready responsibilities that medallion architecture teaches.

Decision and trade-offs. Retaining workload-specific parsers increased maintenance but protected semantic correctness. Storing normalized structured metrics enabled reliable comparisons, while raw data remained important when a parser or benchmark format needed investigation. The database-backed reporting layer improved usability but created a schema contract between parsers and dashboards.

Senior/Staff interview framing.

- **Senior:** show the schema and quality checks needed to turn one raw benchmark result into a comparable record, including run configuration and parser failure handling.
- **Staff:** explain data ownership, lineage from dashboard value back to raw artifact and configuration, schema evolution across benchmark families, and how replayable raw data reduces migration risk.

Scenario 2: truth-bearing artifacts to an evaluated RAG index

Project scenario. DPDK BenchOps Copilot ingested benchmark logs, database JSON, AMD tuning documents, methodology guides, historical run metadata, and internal notes. It normalized and phase-chunked the content, attached workload/platform/source/provenance metadata, retained authoritative records and artifacts in Postgres and S3/MinIO, and used a vector database for embeddings. CI gates evaluated context precision/recall, groundedness, citation coverage, tool reliability, and p95 latency.

How the lifecycle concepts apply. This is an RAG data-product lifecycle:

```
source artifacts and records
→ normalized, metadata-rich benchmark knowledge
→ semantic index
→ evaluated publication
→ traced query and tool behavior
```

The design distinguishes data preparation from index publication and separates truth storage from retrieval-optimized representations.

Decision and trade-offs. Rich metadata and evaluated publication add pipeline work, but reduce the risk of retrieving the right words for the wrong workload or AMD generation. Separate stores add lineage and synchronization obligations, but permit authoritative records and semantic indexes to evolve according to different access patterns.

Senior/Staff interview framing.

- **Senior:** trace a source version through normalization, chunks, embeddings, retrieval, citation, and a failing evaluation case.
- **Staff:** describe the index as a governed data product with owners, quality gates, lineage, publication, rollback, and freshness expectations—not merely a vector-store write.

Evidence boundary and product gap. Neither project documents Databricks, Spark, Delta Lake, Unity Catalog, MLflow, a feature store, or a model registry. Use these scenarios to explain transferable data-lifecycle principles, not direct experience with those products.

Hypothetical improvement. If the organization later standardized on a lakehouse, it could evaluate Bronze/Silver/Gold tables for benchmark and ingestion lineage and MLflow for tracking RAG/application versions and evaluations. This is a proposed evolution, not an implemented project outcome.

10. Interview questions

1. Data lake versus warehouse versus lakehouse?
2. Workspace, notebook, compute, job, and SQL warehouse roles?
3. ETL versus ELT?
4. Bronze, Silver, and Gold responsibilities?
5. What does ACID add to lakehouse tables?
6. Schema enforcement versus evolution?
7. How do time travel and MERGE help?
8. Why are small files and partition selection operational concerns?
9. Parameters versus metrics versus artifacts?
- 10
 - . Experiment tracking versus registry versus serving versus monitoring?
- 11
 - . How do aliases support promotion and rollback?
- 12
 - . How does MLflow extend to RAG and agents?
- 13
 - . How do Delta Lake, MLflow, Unity Catalog, and Databricks relate?
- 14
 - . How do all-purpose, job, SQL-warehouse, and serverless compute differ?
- 15
 - . When would Auto Loader, COPY INTO, or a managed pipeline fit ingestion?

11. Exit checklist

- [] Draw a lakehouse and medallion pipeline.
- [] Explain Delta transaction/schema/version/incremental behavior.
- [] Design batch and streaming quality controls.
- [] Build reproducible ML and RAG data lineage.
- [] Explain MLflow run, registry, alias, promotion, and rollback.
- [] Assign each data type to the right storage system.
- [] Connect governance across data, models, GenAI, and applications.
- [] Select compute and ingestion mechanisms from workload and operational requirements.

Source notes

- [Databricks Fundamentals](#)
- [Delta Lake ETL/ELT](#)
- [MLflow Lifecycle](#)
- [MLOps for AI Systems](#)
- [Databases for AI Systems](#)
- [IBM IJP Roles](#)
- [Capstone Revision Day 1](#)
- [DPDK Automation for Network Packet Processing](#)
- [DPDK BenchOps Copilot](#)